

Simulacija naelektrenih delcev v 2D prostoru

Vzporedno, porazdeljeno in sekvenčno računalništvo – Študijsko gradivo

Gregor Antonaz (89201088)

Fakulteta za računalništvo in informatiko

Junij 2026

Kazalo

1	Uvod v vzporedno in porazdeljeno računalništvo	2
1.1	Niti proti procesom in modeli pomnilnika	2
1.2	Amdahlov zakon	2
1.3	MESI protokol in koherentnost predpomnilnikov	3
2	Pregled fizikalnega problema in rešitve	4
2.1	Coulombov zakon z obrnjeno logiko	4
2.2	Mejna sila in Eulerjeva integracija	4
3	Pregled kode po razredih	5
3.1	Particle.java – Podatkovni model	5
3.2	ForceKernel.java – Skupno računsko jedro	5
3.3	Simulation in AbstractSimulation – skupna osnova	7
3.4	SequentialSimulation.java – Sekvenčna različica	7
3.5	ParallelSimulation.java – Vzporedna različica	8
3.6	DistributedSimulation.java – Porazdeljena različica z MPI	9
3.7	Skupna simulacijska zanka	10
4	Zakaj je vzporedna hitrejša od porazdeljene na enem računalniku	11
4.1	Cena sinhronizacije	11
4.2	Trend je pomembnejši od posamezne številke	11
4.3	Kdaj se porazdeljeni model izplača	11
4.4	Zakaj pohitritev ne doseže $10\times$	11
5	Možna vprašanja na zagovoru in odgovori	13

1 Uvod v vzporedno in porazdeljeno računalništvo

Sodobni računalniški sistemi temeljijo na arhitekturah, ki za izboljšanje zmogljivosti ne povečujejo več frekvence procesorskih jeder (zaradi termičnih omejitev), temveč uvajajo večjedrnost. Za učinkovito izrabo teh jeder moramo programe zasnovati vzporedno ali porazdeljeno.

1.1 Niti proti procesom in modeli pomnilnika

Dve osnovni paradigmi za izvajanje vzporednih izračunov sta:

1. Vzporedno računalništvo z deljenim pomnilnikom (Shared Memory):

- Uporablja **niti (Threads)** znotraj enega samega operacijskega procesa.
- Vse niti delijo enak naslovni prostor (Heap pomnilnik), kar pomeni, da lahko neposredno berejo in pišejo v iste spremenljivke in objekte.
- Komunikacija med nitmi je implicitna in izjemno hitra.
- Glavna slabost so **tekme za vire (Race Conditions)**, ki zahtevajo sinhronizacijo (npr. ključavnice, semaforji, bariere), ter težave s koherentnostjo predpomnilnikov.

2. Porazdeljeno računalništvo s prenosom sporočil (Message Passing / Shared Nothing):

- Uporablja ločene **procese (Processes)**, ki tečejo v svojih izoliranih pomnilniških prostorih, pogosto na različnih fizičnih računalnikih (gručah).
- Procesi ne morejo neposredno dostopati do pomnilnika drug njegova.
- Vsa komunikacija in usklajevanje poteka preko **eksplicitnega pošiljanja sporočil** preko omrežja (npr. standard MPI - Message Passing Interface).
- Prednost je enostavno skaliranje na poljubno število strojev in odsotnost klasičnih pomnilniških tekmovalij za vire.
- Slabost je velik komunikacijski režijski strošek (network latency/bandwidth bound) in kompleksnejše programiranje.

1.2 Amdahlov zakon

Maksimalno teoretično pohitritev vzporednega programa določa **Amdahlov zakon**:

$$S(P) = \frac{1}{s + \frac{1-s}{P}} \quad (1)$$

kjer je:

- $S(P)$ pohitritev z uporabo P procesnih enot.
- s delež programa, ki se mora izvajati strogo sekvenčno (npr. inicializacija, branje datotek, sinhronizacijske bariere).
- $1 - s$ delež programa, ki ga je mogoče popolnoma paralelizirati.

Zakon nam pove, da sekvenčni del programa predstavlja absolutno mejo pohitritve. Če je 10% programa sekvenčnega ($s = 0.1$), program ne more dosegati več kot 10-kratne pohitritve, čeprav bi uporabili neskončno število procesorjev. V naši simulaciji je sekvenčni del posodobitve koordinat $O(n)$ zelo majhen v primerjavi s kvadratnim računanjem sil $O(n^2)$, wat omogoča odlično skaliranje.

1.3 MESI protokol in koherentnost predpomnilnikov

Ker ima vsako procesorsko jedro svoj lokalni predpomnilnik (L1 in L2 Cache), se pri deljenem pomnilniku pojavi problem koherentnosti. Če jedro A spremeni podatke na naslovu X, mora jedro B to zaznati. V ta namen se uporablja protokol **MESI** (Modified, Exclusive, Shared, Invalid):

- **M (Modified)**: Podatek v predpomnilniku je posodobljen in se razlikuje od tistega v glavnem pomnilniku (DRAM). Jedro ima edino veljavno kopijo.
- **E (Exclusive)**: Podatek je enak kot v DRAM-u in je prisoten le v tem predpomnilniku.
- **S (Shared)**: Podatek je enak kot v DRAM-u in je prisoten v predpomnilnikih več jeder.
- **I (Invalid)**: Podatek v predpomnilniku ni več veljaven (drugo jedro ga je spremenilo).

Če več niti na različnih jedrih piše v spremenljivke, ki ležijo na isti predpomnilniški vrstici (običajno 64 bajtov), pride do pojava **lažnega deljenja (False Sharing)**. Jedra morajo neprestano invalidirati predpomnilnike drugih jeder, kar drastično upočasni izvajanje programa.

2 Pregled fizikalnega problema in rešitve

Projekt simulira fizikalni sistem n naelektrenih delcev v dvodimenzionalnem prostoru z dimenzijami $[minX, maxX] \times [minY, maxY]$.

2.1 Coulombov zakon z obrnjeno logiko

Sila med dvema delcema i in j se izračuna po modificiranem Coulombovem zakonu:

$$F_{ij} = \frac{|c_i \cdot c_j|}{d_{ij}^2} \quad (2)$$

kjer je:

- c_i, c_j električni naboj delcev.
- d_{ij} Evklidska razdalja med delcema: $d_{ij} = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$.

Ključni koncept

Obrnjena fizikalna logika (Andrews, poglavje 11.2.1): Za razliko od dejanskih zakonov narave, v tem projektu velja naslednje:

- Delca z **enakim predznakom** naboja se **privlačita** (sila vleče delec proti drugemu).
- Delca z **različnim predznakom** naboja se **odbijata** (sila potiska delec v nasprotno smer).

To je pomembna podrobnost, ki jo je treba izpostaviti na zagovoru.

Da bi se izognili neskončno velikim silam ob morebitnem trku delcev (ko razdalja teži proti 0), je razdalja omejena na spodnjo mejo 1.0 slikovne pike:

$$d_{ij}^2 = \max(1.0, (x_j - x_i)^2 + (y_j - y_i)^2) \quad (3)$$

2.2 Mejna sila in Eulerjeva integracija

Da delci ne bi zapustili simulacijskega okna, nanje deluje odbojna sila robov, če se jim približajo na manj kot 5.0 enot:

$$F_{\text{meja}} = \frac{10.0}{d_{\text{meja}}^2} \quad (4)$$

Premik delcev poteka z **Eulerjevo integracijo** v dveh fazah. Najprej se za vse delce izračunajo rezultantne sile ($O(n^2)$), šele nato se posodobijo stanja ($O(n)$):

$$a_i = \frac{F_i}{m_i}, \quad v'_i = v_i + a_i, \quad x'_i = x_i + v'_i \quad (5)$$

Masa delcev m_i je za vse delce fiksirana na 1.0, kar poenostavi Newtonov zakon ($a_i = F_i$).

3 Pregled kode po razredih

V tem poglavju je predstavljena podrobna struktura in koda ključnih razredov projekta s pripadajočimi teoretičnimi utemeljitvami.

3.1 Particle.java – Podatkovni model

Razred `Particle` je model delca, ki shranjuje koordinate, hitrosti, naboj in maso. Ključna je statična metoda `randomParticle`, ki z determinističnim generatorjem `Random` zagotavlja ponovljive rezultate.

```

1 package com.example.chargedparticles.model;
2 import java.util.Random;
3
4 public class Particle {
5     private double x, y;           // Pozicija
6     private double vx, vy;         // Hitrost
7     private double charge;         // Naboj
8     private double mass = 1.0;
9
10    public Particle(double x, double y, double vx, double vy, double
11    charge) {
12        this.x = x; this.y = y; this.vx = vx; this.vy = vy; this.charge
13        = charge;
14    }
15
16    public static Particle randomParticle(double minX, double maxX,
17    double minY, double maxY, Random random) {
18        double x = minX + random.nextDouble() * (maxX - minX);
19        double y = minY + random.nextDouble() * (maxY - minY);
20        double vx = random.nextDouble() * 2.0 - 1.0;
21        double vy = random.nextDouble() * 2.0 - 1.0;
22        double charge = random.nextBoolean() ? 0.5 + random.nextDouble()
23        : -(0.5 + random.nextDouble());
24        return new Particle(x, y, vx, vy, charge);
25    }
26    // Getterji in setterji...
27 }

```

Listing 1: Koda razreda `Particle.java` (delna)

3.2 ForceKernel.java – Skupno računsko jedro

Razred `ForceKernel` vsebuje **vso fiziko** simulacije. Uporabljajo ga vsi trije načini, zato so enačbe zapisane na enem samem mestu.

Delci niso shranjeni kot objekti, ampak v **ravnem polju** `double[]`, po 6 vrednosti na delec: `[x, y, vx, vy, charge, mass]`. Jedro ponuja dve operaciji, ki obe delujeta na poljubnem območju `[start, end)`:

```

1 package com.example.chargedparticles.simulation;
2
3 public final class ForceKernel {
4
5     public static final int FIELDS_PER_PARTICLE = 6;
6     private static final double BUFFER = 5.0;
7     private static final double REPEL_FACTOR = 10.0;
8
9     // Izracuna sile za delce [start, end) proti vsem n delcem.

```

```

10     public static void computeForces(double[] state, int n, int start,
11     int end,
12                                     double[] forces,
13     SimulationParameters params) {
14         for (int i = start; i < end; i++) {
15             int oi = i * FIELDS_PER_PARTICLE;
16             double x1 = state[oi], y1 = state[oi + 1], c1 = state[oi +
17             4];
18             double fxSum = 0.0, fySum = 0.0;
19
20             for (int j = 0; j < n; j++) {
21                 if (j == i) continue;
22                 int oj = j * FIELDS_PER_PARTICLE;
23                 double dx = state[oj] - x1;
24                 double dy = state[oj + 1] - y1;
25                 double c2 = state[oj + 4];
26
27                 double r2 = dx * dx + dy * dy;
28                 if (r2 < 1.0) r2 = 1.0; // preprecimo eksplozijo sile
29                 double r = Math.sqrt(r2);
30
31                 double magnitude = Math.abs(c1 * c2) / r2;
32                 // Enak predznak = privlacenje, razlicen = odbijanje
33                 double sign = (c1 * c2 >= 0) ? 1.0 : -1.0;
34
35                 fxSum += sign * magnitude * (dx / r);
36                 fySum += sign * magnitude * (dy / r);
37             }
38             // ... mejne sile ...
39             forces[i * 2] = fxSum; // globalni indeks!
40             forces[i * 2 + 1] = fySum;
41         }
42     }
43
44     // Posodobi hitrosti in pozicije za delce [start, end).
45     public static void integrate(double[] state, int start, int end,
46     double[] forces) {
47         for (int i = start; i < end; i++) {
48             int o = i * FIELDS_PER_PARTICLE;
49             double mass = state[o + 5];
50             double newVx = state[o + 2] + forces[i * 2] / mass;
51             double newVy = state[o + 3] + forces[i * 2 + 1] / mass;
52             state[o] += newVx;
53             state[o + 1] += newVy;
54             state[o + 2] = newVx;
55             state[o + 3] = newVy;
56         }
57     }
58 }

```

Listing 2: Fizikalna logika v razredu ForceKernel.java (delna)

Ključni koncept: globalno indeksiranje sil

Rezultat za delec i gre **vedno** v `forces[i*2]` in `forces[i*2+1]`, ne glede na to, kateri izvajalec ga je izračunal. Ker so območja različnih izvajalcev disjunktna, lahko več niti hkrati piše v isto polje **brez zaklepanja**. Prav ta lastnost omogoča, da imajo vsi trije načini isto jedro.

3.3 Simulation in AbstractSimulation – skupna osnova

Vmesnik `Simulation` predpisuje tri metode, `AbstractSimulation` pa hrani stanje, ki je pri vseh treh načinih enako.

```

1 public interface Simulation {
2     void performOneCycle();
3     void syncToParticles(List<Particle> particles);
4     String getDescription();
5     default void shutdown() { }
6 }
7
8 public abstract class AbstractSimulation implements Simulation {
9     protected final int n;           // stevilo delcev
10    protected final double[] state;   // 6 doublov na delec
11    protected final double[] forces;  // forces[i*2], forces[i*2+1]
12    protected final SimulationParameters params;
13
14    protected AbstractSimulation(List<Particle> particles,
15    SimulationParameters params) {
16        this.n = particles.size();
17        this.state = ForceKernel.toFlatArray(particles);
18        this.forces = new double[n * 2];
19        this.params = params;
20    }
21
22    @Override
23    public void syncToParticles(List<Particle> particles) {
24        ForceKernel.writeBack(state, particles);
25    }
26 }

```

Listing 3: Vmesnik in skupna osnova

Simulacija računa nad ravnim poljem, zato seznam `Particle` objektov med izvajanjem ni ažuren. Metodo `syncToParticles()` kličemo samo pred izrisom ali izpisom, nikoli v vroči zanki.

3.4 SequentialSimulation.java – Sekvenčna različica

Najenostavnejša različica: ena nit obdela celotno območje $[0, n)$. Služi kot osnova za primerjave.

```

1 public class SequentialSimulation extends AbstractSimulation {
2
3     public SequentialSimulation(List<Particle> particles,
4     SimulationParameters params) {
5         super(particles, params);
6     }
7
8     @Override
9     public void performOneCycle() {
10        ForceKernel.computeForces(state, n, 0, n, forces, params);
11        ForceKernel.integrate(state, 0, n, forces);
12    }
13 }

```

```

11     }
12
13     @Override
14     public String getDescription() { return "Sekvencna"; }
15 }

```

Listing 4: Celotna sekvencna implementacija

3.5 ParallelSimulation.java – Vzporedna različica

Delce razdeli na T enako velikih odsekov; vsak odsek izračuna svoja nit iz thread poola. Naloge so vedno enake, zato jih zgradimo **enkrat** v konstruktorju.

```

1 public class ParallelSimulation extends AbstractSimulation {
2
3     private final int numThreads;
4     private final ExecutorService executor;
5     private final List<Callable<Void>> tasks = new ArrayList<>();
6
7     public ParallelSimulation(List<Particle> particles,
8 SimulationParameters params,
9                             int numThreads) {
10         super(particles, params);
11         this.numThreads = numThreads;
12         this.executor = Executors.newFixedThreadPool(numThreads);
13
14         int chunk = (n + numThreads - 1) / numThreads;
15         for (int t = 0; t < numThreads; t++) {
16             final int start = Math.min(t * chunk, n);
17             final int end = Math.min(start + chunk, n);
18             if (start < end) {
19                 tasks.add(() -> {
20                     ForceKernel.computeForces(state, n, start, end,
21 forces, params);
22                     return null;
23                 });
24             }
25         }
26
27         @Override
28         public void performOneCycle() {
29             // FAZA 1: vzporeden izracun sil; invokeAll pocaka na vse niti.
30             try {
31                 executor.invokeAll(tasks);
32             } catch (InterruptedException e) {
33                 Thread.currentThread().interrupt();
34                 return;
35             }
36             // FAZA 2: posodobitev pozicij - O(n), zanemarljivo glede na O(n
37             ~2).
38             ForceKernel.integrate(state, 0, n, forces);
39         }
40     }
41 }

```

Listing 5: Delitev dela in pregrada v ParallelSimulation.java

Zakaj invokeAll namesto CountdownLatch

invokeAll() odda vse naloge in se vrne šele ko so vse končane – že sam po sebi je pregrada (barrier). Ročnega števca torej ni treba ustvarjati, zmanjševati in čakati nanj. Manj kode pomeni manj možnosti za napako: pozabljen countdown() v veji z izjemo bi glavno nit blokiral za vedno.

3.6 DistributedSimulation.java – Porazdeljena različica z MPI

Vsak proces hrani kopijo stanja **vseh** delcev, računa pa samo svoj odsek. Na koncu cikla Allgatherv združi odseke nazaj v celotno stanje na vseh procesih.

```

1 public class DistributedSimulation extends AbstractSimulation {
2
3     private final int rank, size, myStart, myEnd;
4     private final int[] recvCounts, displacements;
5
6     public DistributedSimulation(List<Particle> particles,
7     SimulationParameters params,
8         int rank, int size) {
9         super(particles, params);
10        this.rank = rank;
11        this.size = size;
12        this.myStart = startIndexOf(rank);
13        this.myEnd = startIndexOf(rank + 1);
14
15        this.recvCounts = new int[size];
16        this.displacements = new int[size];
17        for (int r = 0; r < size; r++) {
18            int start = startIndexOf(r);
19            recvCounts[r] = (startIndexOf(r + 1) - start) *
20            ForceKernel.FIELDS_PER_PARTICLE;
21            displacements[r] = start * ForceKernel.FIELDS_PER_PARTICLE;
22        }
23
24        // Prvih (n mod size) procesov dobi en delec vec.
25        // Za r == size vrne n, zato je konec odseka r kar startIndexOf(r +
26        // 1).
27        private int startIndexOf(int r) {
28            int base = n / size;
29            int remainder = n % size;
30            return (r < remainder) ? r * (base + 1)
31                : remainder * (base + 1) + (r - remainder
32                ) * base;
33        }
34
35        @Override
36        public void performOneCycle() {
37            // FAZA 1 + 2: izracun sil in novih pozicij za lasten odsek.
38            ForceKernel.computeForces(state, n, myStart, myEnd, forces,
39            params);
40            ForceKernel.integrate(state, myStart, myEnd, forces);
41
42            // FAZA 3: vsak proces poslje svoj odsek in prejme odseke vseh
43            // ostalih.
44            int sendOffset = myStart * ForceKernel.FIELDS_PER_PARTICLE;

```

```

40     int sendCount = (myEnd - myStart) * ForceKernel.
        FIELDS_PER_PARTICLE;
41     try {
42         MPI.COMM_WORLD.Allgatherv(
43             state, sendOffset, sendCount, MPI.DOUBLE,
44             state, 0, recvCounts, displacements, MPI.DOUBLE);
45     } catch (MPIException e) {
46         throw new IllegalStateException("Napaka pri MPI komunikaciji
        ", e);
47     }
48 }
49 }

```

Listing 6: Delitev delcev in kolektivna sinhronizacija

3.7 Skupna simulacijska zanka

Vsi trije načini tečejo po isti zanki, zapisani enkrat v `SimulationRunner`:

```

1 public static void runCycles(Simulation simulation, List<Particle>
    particles,
2                               SimulationParameters params, boolean
    printResults) {
3     long startTime = System.nanoTime();
4
5     for (int cycle = 0; cycle < params.getNumCycles(); cycle++) {
6         if (Thread.currentThread().isInterrupted()) return;
7         simulation.performOneCycle();
8
9         // Za izris moramo stanje prepisati v Particle objekte.
10        if (params.isEnableUI()) {
11            simulation.syncToParticles(particles);
12            try { Thread.sleep(UI_FRAME_DELAY_MS); }
13            catch (InterruptedException e) { Thread.currentThread().
    interrupt(); return; }
14        }
15    }
16    // ... izpis casa in prvih petih delcev, ce printResults ...
17 }

```

Listing 7: `SimulationRunner.runCycles` – uporabljajo jo vsi trije načini

4 Zakaj je vzporedna hitrejša od porazdeljene na enem računalniku

Primerjava izvedbenih modelov je smiselna **samo**, če vsi izvajajo isti algoritem. Če bi se različice razlikovale v notranji zanki ali v načinu shranjevanja delcev, bi izmerjena razmerja odražala te razlike in ne učinka paralelizacije. Zato vsi trije načini kličejo **isto jedro nad isto podatkovno postavitvijo** in tečejo v **isti zanki**. Edina razlika je, kdo izračuna kateri odsek in kako se sinhronizira na koncu cikla.

4.1 Cena sinhronizacije

Obe vzporedni različici razdelita enako delo med enako število izvajalcev in se enkrat na cikel sinhronizirata. Razlikuje se **cena** te sinhronizacije:

- **Niti** si delijo eno polje. Ko je pregrada dosežena, vsaka nit **že vidi** posodobljeno stanje – prekopira se nič.
- **MPI procesi** imajo vsak svojo kopijo. Vsak cikel se konča z **Allgatherv**, ki prenese $6 \cdot 8 \cdot n = 48n$ bajtov na proces. Pri $n = 2000$ je to 96 kB na proces, in to **v vsakem ciklu**.

Nobeno od tega kopiranja ne pripomore k izračunu – obstaja samo zato, ker procesi ne vidijo pomnilnika drug drugega. Zato je na enem računalniku vzporedna različica **vedno** hitrejša, in to je pričakovan rezultat.

4.2 Trend je pomembnejši od posamezne številke

Komunikacija raste kot $O(n)$, računanje pa kot $O(n^2)$. Delež časa, porabljen za komunikacijo, mora zato z večanjem problema **padati** – in meritve to potrjujejo:

n	Vzporedna [s]	Porazdeljena $P=10$ [s]	Porazdeljena počasnejša za
500	0,161	0,608	278 %
1000	0,467	0,798	71 %
2000	1,453	2,386	64 %
3000	4,159	5,416	30 %

Pri $n = 500$ je porazdeljena različica z 10 procesi celo **počasnejša od sekvenčne** (pohitritev $0,72\times$): računanja na cikel je tako malo, da 1000 klicev **Allgatherv** prevlada nad celotnim časom izvajanja. To je najlepši primer v naših podatkih, da paralelizacija ni brezplačna.

4.3 Kdaj se porazdeljeni model izplača

Porazdeljena različica plačuje za **zmožnost**, ki je en računalnik ne potrebuje: delovanje v ločenih naslovnih prostorih. Njena prednost se pokaže šele, ko procesi tečejo na **različnih računalnikih** – takrat sta na voljo skupni pomnilnik in skupno število jeder več strojev. Znotraj enega računalnika te prednosti ni, zato je komunikacija čisti režijski strošek.

4.4 Zakaj pohitritev ne doseže $10\times$

Apple M4 ima **4 zmogljiva (P) in 6 varčnih (E) jeder**. Prvi štirje izvajalci dobijo P-jedra in prispevajo skoraj polno zmogljivost; naslednjih šest dobi E-jedra, ki so bistveno počasnejša. Pohitritev zato še naprej raste, učinkovitost na izvajalca pa pada:

Nastavitev	Pohitritev	Učinkovitost
MPI, 4 procesi	2,0–2,9×	51–73 %
MPI, 10 procesov	2,2–2,9×	22–29 %
Vzporedna, 10 niti	2,7–4,8×	27–48 %

To še poslabšata dve lastnosti izvedbe: kosi so **enako veliki**, na koncu vsakega cikla pa je **pregrada**. Izvajalec na E-jedru dobi enako dela kot tisti na P-jedru, a ga konča kasneje – in vsi ostali čakajo nanj. Kose bi bilo mogoče razdeliti sorazmerno z zmogljivostjo jedra ali uvesti dinamično dodeljevanje dela, s čimer bi del te izgube povrnili.

Sklep

Na enem računalniku je vrstni red vedno enak: **vzporedna** > **porazdeljena** > **sekvenčna**. Porazdeljena različica zaostaja, ker vsak cikel prekopira celotno stanje med procesi; njen zaostanek pa z večanjem problema pada, ker računanje raste hitreje od komunikacije.

5 Možna vprašanja na zagovoru in odgovori

Ta razdelek vsebuje reprezentativna teoretična in praktična vprašanja v slovenščini, s katerimi se lahko študent sreča na ustnem zagovoru projekta pri Programiranju 3.

Možno vprašanje na zagovoru

Vprašanje 1: Kakšna je razlika med vzporednim (Parallel) in porazdeljenim (Distributed) načinom delovanja v vaši implementaciji?

Odgovor: Vzporedni način uporablja **večnitni model z deljenim pomnilnikom**. Teče znotraj ene same Java virtualne mašine (JVM), kjer niti (`ExecutorService`) neposredno berejo in pišejo v **isto ravno polje state**. Prekopira se nič – ko je pregrada (`invokeAll`) dosežena, vsaka nit že vidi posodobljeno stanje.

Porazdeljeni način uporablja **model s prenosom sporočil (Message Passing)** preko MPJ Express. Teče kot več samostojnih, izoliranih operacijskih procesov, ki ne delijo pomnilnika (Shared Nothing). Vsak proces ima svojo kopijo sploščenega polja **state**. Medsebojna izmenjava podatkov poteka eksplicitno preko kolektivne komunikacije `MPI.COMM_WORLD.Allgatherv` po zaključku vsakega cikla.

Možno vprašanje na zagovoru

Vprašanje 2: Zakaj je v simulacijski zanki uporabljen dvofazni algoritem (ločen izračun sil in ločena posodobitev pozicij)?

Odgovor: Če bi pozicijo in hitrost delca posodobili takoj, ko za njega izračunamo silo, bi naslednji delec v istem ciklu pri računanju sile “videl” že posodobljene koordinate tega delca namesto začetnih. To bi povzročilo fizikalno neskladnost in asimetrijo (akcija ne bi bila enaka reakciji, saj bi se sile računale na različnih koordinatnih stanjih). Z dvofaznim pristopom zagotovimo, da vsi delci v fazi 1 vidijo popolnoma enako, statično stanje vesolja iz prejšnjega cikla, v fazi 2 pa vsi hkrati posodobijo svoje koordinate.

Možno vprašanje na zagovoru

Vprašanje 3: Zakaj je vzporedna različica paralelizirana samo v Fazi 1 (izračun sil), Faza 2 (Eulerjeva integracija) pa se izvaja sekvenčno?

Odgovor: To odločitev utemeljuje časovna kompleksnost obeh faz. Izračun sil (Faza 1) is časovno izjemno potraten, saj zahteva računanje interakcij vsakega delca z vsakim drugim, kar ima kvadratno časovno kompleksnost $O(n^2)$. Za 3000 delcev je to 9.000.000 interakcij. Posodobitev pozicij (Faza 2) pa je linearna operacija $O(n)$, saj preteče vsak delec le enkrat. Za 3000 delcev je to le 3000 enostavnih matematičnih operacij, kar se na sodobnem procesorju izvede v nekaj mikrosekundah. Paralelizacija faze 2 bi zaradi režijskega stroška ustvarjanja nalog in sinhronizacije niti program dejansko upočasnila.

Možno vprašanje na zagovoru

Vprašanje 4: Kako ste v vzporedni različici ParallelSimulation zagotovili varnost pred tekmo za vire (Thread Safety) brez uporabe počasnih ključavnic (locks) ali sinhroniziranih blokov?

Odgovor: Uporabili smo princip **prostorske dekompozicije tabele sil**. Pred začetkom vzporednega dela alociramo skupno polje sil `forces[n][2]`. Vsaki nit t dodelimo natanko določen, disjunkten razpon delcev (npr. od indeksa `start` do `end`). Ker vsaka nit izračunava sile in jih zapisuje **izključno v svoje vrstice** znotraj polja `forces` (na indekse od `start` do `end`), med njimi nikoli ne pride do prekrivanja pri pisanju. Ker ni deljenih pisalnih mest, zaklepanje ali uporaba sinhroniziranih zbirk sploh nista potrebna, kar omogoča maksimalno hitrost izvajanja brez blokad.

Možno vprašanje na zagovoru

Vprašanje 5: Pojasnite, kako deluje kolektivna operacija `Allgatherv` v vaši MPI implementaciji.

Odgovor: `Allgatherv` je kolektivna komunikacijska operacija tipa “vsi zberejo vse, z variabilnimi dolžinami”. Vsak MPI proces ima svoj podnabor lokalno posodobljenih delcev. Ko vsi procesi pokličejo `Allgatherv`, se zgodi naslednje: 1. Vsak proces pošlje svoj posodobljeni segment koordinat iz svojega polja `state` (določen s `sendOffset` in `sendCount`). 2. Hkrati vsak proces sprejme posodobljene segmente od vseh ostalih procesov. 3. Ker se lahko število delcev na proces razlikuje, operacija uporablja polje `recvCounts` (koliko doublov prejmemo od posameznega ranga) in `displacements` (na katero indeksno mesto v polju se zapišejo podatki posameznega ranga). Po končanem klicu imajo vsi procesi v svojem lokalnem pomnilniku popolnoma enak in posodobljen celotni niz `state`.

Možno vprašanje na zagovoru

Vprašanje 6: Zakaj so podatki delcev shranjeni v ravno polje `double[]` in ne kot objekti tipa `Particle`?

Odgovor: Razlogi so trije, zadnji pa je najpomembnejši za to poročilo. 1. **Serializacija:** Standardno pošiljanje Java objektov zahteva serializacijo (prebiranje metapodatkov objekta, pretvorba v bajte in rekonstrukcija na ciljnem procesu), kar je drago. Pošiljanje ploščatega polja primitivnih `double`-ov (`MPI.DOUBLE`) poteka brez kakršne koli pretvorbe – MPI preprosto prekopira neprekinjen blok pomnilnika. 2. **Predpomnjenje:** Objekti `Particle` so razpršeni po kopici, zato branje seznama pomeni sledenje smernikom (pointer chasing) in pogoste zgrešitve predpomnilnika. Ravno polje je en sam neprekinjen blok, kjer strojno preddobivanje (Hardware Prefetcher) zazna linearen vzorec in podatke naloži vnaprej. 3. **Poštena primerjava:** Polje uporabljajo **vsi trije načini**, ne samo MPI. Če bi sekvenčna in vzporedna različica delali nad objekti, porazdeljena pa nad ravnim poljem, bi izmerjena razmerja odražala **razliko v podatkovni postavitvi** in ne učinka paralelizacije. Z enotno postavitvijo merimo to, kar želimo meriti.

Možno vprašanje na zagovoru

Vprašanje 7: Kaj je “lažno deljenje” (False Sharing) in ali se lahko pojavi v vaši `ParallelSimulation` različici?

Odgovor: Lažno deljenje se pojavi, ko dve različni jedri na svojih lokalnih predpomnilnikih (L1/L2 Cache) spreminjata neodvisni spremenljivki, ki pa se nahajata znotraj **iste predpomnilniške vrstice** (običajno 64 bajtov). Ko jedro A piše v svojo spremenljivko, strojna oprema preko MESI protokola invalidira celotno vrstico v predpomnilniku jedra B. Jedro B mora nato znova brati podatek iz počasnejšega skupnega predpomnilnika L3 ali DRAM-a, čeprav piše v čisto drugo spremenljivko.

V naši `ParallelSimulation` se to teoretično lahko pojavi na **mejah kosov** (chunk boundaries) znotraj polja `forces[n]` [2]. Če se konec kosa za nit 0 in začetek kosa za nit 1 nahajata v isti 64-bajtni vrstici, bo prišlo do lažnega deljenja. Ker pa so naši kosi relativno veliki (npr. pri 1000 delcih in 4 nitmi je chunk size $250 \text{ delcev} \times 16 \text{ bajtov} = 4000 \text{ bajtov}$), se lažno deljenje pojavi le na stičnih točkah in ne vpliva resneje na celotno zmogljivost.

Možno vprašanje na zagovoru

Vprašanje 8: Pojasnite vzorca načrtovanja Strategy in Factory, ki sta uporabljena v vaši kodi.

Odgovor: 1. **Strategy vzorec:** Definiran je vmesnik `Simulation`, ki predpisuje metodo `performOneCycle`. Razreda `SequentialSimulation` in `ParallelSimulation` implementirata ta vmesnik. Glavni razred `SimulationRunner` drži referenco tipa `Simulation` in sploh ne ve (niti ga ne zanima), katera implementacija se dejansko izvaja – le kliče metodo. To omogoča enostavno zamenjavo algoritma računanja v teku programa. 2. **Factory vzorec:** Razred `SimulationFactory` vsebuje statično metodo `createSimulation(SimulationMode mode)`. Na podlagi parametra (enum `SimulationMode`) ustvari in vrne ustrezen objekt (npr. `new ParallelSimulation()`). S tem skrijemo kompleksnost kreiranja objektov pred klicateljem.

Možno vprašanje na zagovoru

Vprašanje 9: Kako se GUI (grafični vmesnik) odziva, ko se simulacija izvaja z maksimalno hitrostjo, in kako preprečite zamrznitev okna?

Odgovor: Za preprečitev zamrznitve grafičnega vmesnika uporabljamo **večnitnost (Thread Separation)**. Izrisovanje GUI-ja poteka strogo na niti za obdelavo grafičnih dogodkov (**Swing Event Dispatch Thread - EDT**). Celotna simulacijska zanka pa se poganja na **ločeni ozadnji niti** (`simulationThread`), ki jo ustvari `SimulationRunner`. EDT ne opravlja nobenih fizikalnih izračunov, temveč le periodično (npr. 60-krat na sekundo, na podlagi proženja `javax.swing.Timer`-ja) izvede ponovni izris trenutnih koordinat delcev s klicem `repaint()`. Simulacijska nit pa nemoteno teče z maksimalno hitrostjo v ozadju. Tako vmesnik ostane popolnoma odziven za gumbe in vnosna polja.

Možno vprašanje na zagovoru

Vprašanje 10: Kaj se zgodi, ko v GUI kliknete gumb “Start” pri izbranem “distributed” načinu? Pojasnite mehanizem zagona.

Odgovor: Porazdeljene simulacije ni mogoče zagnati kot novo nit znotraj trenutne JVM: MPI okolje mora obstajati **že ob zagonu programa**, saj je `MPI.Init()` prvi klic v `DistributedRunner`. Obstoječi proces se ne more sam “razmnožiti” v MPI skupino. Zato `SimulationUI` zazene nov proces preko **ProcessBuilder**:

1. Ob izbiri načina **Porazdeljena (MPI)** se v vmesniku pokaže dodatno polje **Procesi**.

2. Klik na **Start** pokaže potrditveno okno z **točnim ukazom**, ki bo izveden:

```
./run_distributed.sh [P] --particles [N] --cycles [C] --ui true
```

3. Po potrditvi `ProcessBuilder` zazene to skripto. Skripta poišče `JAVA_HOME` in `MPJ_HOME`, prevede projekt in ga pozene z `mpjrun.sh` v `P` procesih.
4. Proces z rangom 0 odpre **svoje okno** in izpisuje rezultate; ostali procesi računajo brez izrisa.

Izhod skripte je preusmerjen v konzolo (`inheritIO()`), tako da so morebitne napake takoj vidne.

Možno vprašanje na zagovoru

Vprašanje 11: Kakšne so fizikalne omejitve, ki preprečijo, da bi se simulacija zrušila ali postala nestabilna?

Odgovor: Glavna stabilizacijska mehanizma sta dva: 1. **Omejitev razdalje med delci (Cap):** V razredu `ForceKernel` ob računanju sil fiksiramo kvadrat razdalje r^2 na najmanj 1.0. Če bi se delca preveč približala (npr. $r = 0.01$), bi po Coulombovem zakonu ($F \propto 1/r^2$) sila eksplodirala do ogromnih vrednosti. Delec bi dobil neskončen pospešek in bi v naslednjem ciklu poletel izven koordinatnega prostora. 2. **Mejna odbojna sila (Boundary Force):** Ko se delec približa robu na manj kot 5 enot, nanj začne delovati eksponentno naraščajoča odbojna sila $F = 10/d^2$, ki ga potisne nazaj. To prepreči uhajanje delcev iz simulacijskega okna.

Možno vprašanje na zagovoru

Vprašanje 12: Zakaj pohitritev ne doseže $10\times$, čeprav ima procesor 10 jeder?

Odgovor: Ker jedra **niso enakovredna**. Apple M4 ima **4 zmogljiva (P) in 6 varčnih (E) jeder**. Prvi štirje izvajalci dobijo P-jedra in prispevajo skoraj polno zmogljivost, naslednjih šest pa E-jedra, ki so bistveno počasnejša. Pohitritev zato še naprej raste, učinkovitost na izvajalca pa močno pade: pri 4 procesih dosežemo 51–73 % učinkovitost, pri 10 pa le 22–29 %.

To še poslabšata dve lastnosti izvedbe:

- **Enako veliki kosi:** izvajalec na E-jedru dobi enako dela kot tisti na P-jedru, a ga konča kasneje.
- **Pregrada na koncu vsakega cikla:** vsi čakajo na najpočasnejšega. Pri 1000 ciklih to ceno plačamo 1000-krat.

Rešitev bi bila razdelitev kosov **sorazmerno z zmogljivostjo jedra** ali **dinamično dodeljevanje dela** (work stealing), pri katerem hitrejša jedra prevzamejo več kosov. Na homogenem procesorju so enaki kosi optimalni, na hibridnem P/E procesorju pa ne. Gre za lastnost strojne opreme, ki enako prizadene vzporedno in porazdeljeno različico.